02-17、前后端实现

1、Agent交互sse服务API

实现流式执行Agent任务，前端实时显示进度数据

1、请求信息

> **接口信息**
>
> - 请求方式: POST
> - 请求路径: /agent/execute/stream
> - 功能描述: 实时流式执行Agent任务，通过Server-Sent Events (SSE)返回执行过程
>
> **请求类型**
>
> Content-Type: application/json
>
> **请求参数**
>
> ```
> {
>  "user_input": "查询web测试平台用户注册的接口，生成测试用例",
>  "user_id": "demo_user",
> }
> ```

参数名	类型	必填	长度限制	描述
user_input	string	是	1-1000字符	用户输入的任务描述
user_id	string	否	-	用户ID，用于会话管理，默认为"anonymous"

2、响应格式

> **响应格式**
>
> Content-Type: text/event-stream
>
> 响应采用SSE格式，每条消息格式为：
>
> ```
> data: {"type": "事件类型", "message": "消息内容", ...}
> ```

3、SSE返回数据详细要求

1. start - 任务开始

触发时机: Agent任务开始执行时

```
{
    "type": "start",
    "message": "开始执行Agent任务",
    "session_id": "session_123456",
    "timestamp": 1703123456.789
}
```

字段说明:

- `type`: 固定值 "start"
- `message`: 开始消息
- `session_id`: 会话ID
- `timestamp`: Unix时间戳

---

2. progress - 进度更新

触发时机: 各个执行阶段的进度更新

```
{
    "type": "progress",
    "step": "intent_recognition",        // 当前步骤标识
    "message": "正在进行意图识别...",
    "progress": 10                       // 进度百分比 0-100
}
```

step字段可能值:

- `intent_recognition`: 意图识别阶段
- `task_planning`: 任务规划阶段

---

3. llm_interaction - LLM交互日志

触发时机: 与大语言模型交互时

3.1 LLM请求 (interaction_type: "request")

```
{
    "type": "llm_interaction",
    "interaction_type": "request",
    "message": "调用大模型进行意图识别",
    "data": {
        "model": "deepseek_v3",
        "input": "用户输入内容",
        "tools_count": 9,
        "prompt_type": "意图识别提示词"
    }
}
```

3.2 LLM响应 (interaction_type: "response")

```
{
    "type": "llm_interaction",
    "interaction_type": "response",
    "message": "大模型意图识别完成",
    "data": {
        "model": "deepseek_v3",
        "confidence": 0.85,
        "intent_type": "接口测试用例生成",
        "primary_tool": "complete_case_generation",
        "execution_plan_steps": 3
    }
}
```

3.3 Function Calling请求

```
{
    "type": "llm_interaction",
    "interaction_type": "request",
    "message": "使用Function Calling调用工具: interface_db_query",
    "data": {
        "model": "deepseek_v3",
        "tool_name": "interface_db_query",
        "prompt_preview": "用户需求：查询web测试平台用户注册的接口..."
    }
}
```

3.4 Function Calling响应

```
{
    "type": "llm_interaction",
    "interaction_type": "response",
    "message": "大模型返回工具调用指令",
    "data": {
        "model": "deepseek_v3",
        "tool_name": "interface_db_query",
        "parameters_generated": true,
        "tool_call_success": true
    }
}
```

---

4. intent_result - 意图识别结果

触发时机: 意图识别完成后

```
{
    "type": "intent_result",
    "data": {
        "intent_type": "接口测试用例生成",
        "task_category": "测试用例生成",
        "confidence": 0.85,
        "primary_tool": "complete_case_generation",
        "tool_chain": ["interface_db_query", "complete_case_generation"],
        "is_batch_operation": false,
        "scope": "单个接口",
```

```
        "parameters": {
            "interface_name": "用户注册",
            "project_name": "web测试平台"
        },
        "missing_params": [],
        "execution_plan": [
            {
                "step": 1,
                "action": "查询接口信息",
                "tool_name": "interface_db_query"
            },
            {
                "step": 2,
                "action": "生成测试用例",
                "tool_name": "complete_case_generation"
            }
        ],
        "workflow_type": "sequential",
        "estimated_time": "2-3分钟",
        "risk_level": "low"
    },
    "progress": 30
}
```

---

5. execution_plan - 执行计划

触发时机: 任务规划完成后

```
{
    "type": "execution_plan",
    "data": [
        {
            "step": 1,
            "action": "查询接口信息",
            "tool_name": "interface_db_query"
        },
        {
            "step": 2,
            "action": "生成测试用例",
            "tool_name": "complete_case_generation"
        }
    ],
    "progress": 50
}
```

---

6. step_start - 步骤开始

触发时机: 开始执行某个具体步骤时

```
{
    "type": "step_start",
    "step_info": {
        "step": 1,
        "action": "查询接口信息",
        "tool_name": "interface_db_query"
    },
    "step_number": 1,
    "total_steps": 2,
    "progress": 60
}
```

---

7. tool_interaction - 工具交互日志

触发时机: 工具执行过程中

7.1 工具开始执行 (interaction_type: "start")

```
{
    "type": "tool_interaction",
    "interaction_type": "start",
    "message": "开始执行工具: interface_db_query",
    "data": {
        "tool_name": "interface_db_query",
        "action": "查询接口信息",
        "step_number": 1
    }
}
```

7.2 工具调用 (interaction_type: "call")

```
{
    "type": "tool_interaction",
    "interaction_type": "call",
    "message": "执行工具: interface_db_query",
    "data": {
        "tool_name": "interface_db_query",
        "action": "查询接口信息"
    }
}
```

7.3 工具执行结果 (interaction_type: "result")

```
{
    "type": "tool_interaction",
    "interaction_type": "result",
    "message": "工具执行完成: interface_db_query",
    "data": {
        "tool_name": "interface_db_query",
        "result": "查询到 3 个相关接口",
        "success": true
    }
}
```

7.4 测试用例执行工具特殊格式

```
{
    "type": "tool_interaction",
    "interaction_type": "call",
    "message": "调用测试用例执行工具",
    "data": {
        "tool_name": "case_execution",
        "parameters": {
            "test_cases_count": 5
        }
    }
}
```

7.5 工具执行错误 (interaction_type: "error")

```
{
    "type": "tool_interaction",
    "interaction_type": "error",
    "message": "工具执行失败: interface_db_query",
    "data": {
        "tool_name": "interface_db_query",
        "error": "数据库连接超时"
    }
}
```

---

8. step_complete - 步骤完成

触发时机: 某个步骤执行完成后

```
{
    "type": "step_complete",
    "step_info": {
        "step": 1,
        "action": "查询接口信息",
        "tool_name": "interface_db_query"
    },
    "step_number": 1,
    "result": "步骤 1 执行完成",
    "progress": 75
}
```

---

9. complete - 任务完成

触发时机: 整个Agent任务执行完成

```
{
    "type": "complete",
    "message": "任务执行完成",
    "data": {
        "result": "任务执行完成",
        "intent_result": {
            // 完整的意图识别结果对象
        },
        "execution_time": 1703123456.789,
        "session_id": "session_123456"
    },
    "progress": 100
}
```

---

10. error - 错误信息

触发时机: 执行过程中发生错误

10.1 置信度过低错误

```
{
    "type": "error",
    "message": "意图识别置信度过低，无法执行任务",
    "confidence": 0.3
}
```

10.2 系统执行错误

```
{
    "type": "error",
    "message": "执行失败: 数据库连接异常",
    "error": "ConnectionError: Unable to connect to database"
}
```

2、后端源码

- **main.py**

```
# @Author  : 木森
# @weixin: python771
import uuid
from typing import AsyncGenerator, Dict

from starlette.middleware.cors import CORSMiddleware

from agents.models import deepseek_v3_llm

import asyncio
import json
import time

from fastapi import FastAPI
import uvicorn
from pydantic import BaseModel, Field
```

```python
from fastapi.responses import StreamingResponse
from agents.agent import MemoryHandler, IntentRecognition, ToolExecutor
from prompts.agent_prompt.intent_recognition import intent_recognition_prompt

# 创建一个FastAPI应用
app = FastAPI(title="API测试智能体",
              description="基于AI的API测试用例生成和执行智能体",
              version="1.0.0")

# 允许跨域请求的配置
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


# 定义执行agent请求参数的模型
class StreamRequest(BaseModel):
    user_id: str = Field(default="musen_01", description="用户ID")
    user_input: str = Field(description="用户输入内容")
    # 用户会话的ID，可以为空，表示新的会话
    session_id: str | None = Field(default=None, description="会话ID")


class ApiAgentExecutor:
    """Agent任务执行的处理器"""

    def __init__(self, user_input: str, user_id: str, session_id: str):
        self.user_id = user_id
        self.user_input = user_input
        self.session_id = session_id
        self.memory = MemoryHandler(user_id, session_id)
        self.memory.save_to_memory({
            "role": "user",
            "content": self.user_input,
            "type": "user_input"
        })

    async def execute_agent(self) -> AsyncGenerator[str, None]:
        """执行egent的入口方法"""
        tools_count = 0
        yield self._format_sse_data({
            "type": "start",
            "message": "开始执行Agent任务",
        })
        await asyncio.sleep(0.1)
        # 创建一个意图识别的对象
        intent = IntentRecognition(
            user_input=self.user_input,
            tools_info=ToolExecutor.get_tools_info()
        )
        yield self._format_sse_data({
            "type": "progress",
            "message": "开始进行用户的意图识别",
            "step": "intent_recognition",
```

```
            "progress": 5
        })
        await asyncio.sleep(0.1)
        tools_count += 1
        yield self._format_sse_data({
            "type": "llm_interaction",
            "interaction_type": "request",
            "message": "调用大模型进行意图识别",
            "data": {
                "model": "deepseek_v3",
                "input": self.user_input,
                "tools_count": tools_count,
                "prompt_type": "意图识别提示词"
            }
        })
        await asyncio.sleep(0.1)
        # 进行意图识别
        intent_result = intent.intent_recognition()
        await asyncio.sleep(0.1)
        yield self._format_sse_data({
            "type": "llm_interaction",
            "interaction_type": "response",
            "message": "大模型意图识别完成",
            "data": {
                "model": "deepseek_v3",
                "confidence": intent_result.get('confidence'),
                "intent_type": intent_result.get('intent_type'),
                "primary_tool": intent_result.get('primary_tool'),
                "execution_plan_steps": f"任务执行需要完成的步骤数为:
{len(intent_result.get('execution_plan'))}",
            }
        })
        await asyncio.sleep(0.1)
        # 显示意图是识别的结果
        yield self._format_sse_data({
            "type": "intent_result",
            "data": {
                "intent_type": intent_result.get('intent_type'),
                "task_category": intent_result.get('task_category'),
                "primary_tool": intent_result.get('primary_tool'),
                "confidence": intent_result.get('confidence'),
            },
            "progress": 8
        })
        await asyncio.sleep(0.1)
        self.memory.save_to_memory({
            "role": "assistant",
            "content": intent_result,
            "type": "intent_result"
        })
        # 判断意图识别的可信度
        if intent_result.get('confidence') < 0.7:
            # 直接使用大模型来回复问题
            response = deepseek_v3_llm.invoke(self.user_input)
            yield self._format_sse_data({
                "type": "llm_interaction",
                "interaction_type": "response",
                "message": f"意图识别可信度太低，直接回复问题",
```

```
                "data": response.content,
            })
        else:
            yield self._format_sse_data({
                "type": "progress",
                "step": "task_planning",
                "message": "正在规划任务的执行...",
                "progress": 10
            })
            # 获取任务列表
            execution_plan = intent_result.get('execution_plan')
            yield self._format_sse_data({
                "type": "execution_plan",
                "data": execution_plan,
                "message": "生成任务执行计划",
                "progress": 10
            })
            # 创建一个任务执行器
            tools_execute = ToolExecutor(self.user_input, self.memory,
intent_result, execution_plan[::-1])
            # 保存执行结果的异步生成器
            tools_execute_generator = tools_execute.task_execution()
            # 遍历异步生成器，获取任务执行进度的信息
            async for info in tools_execute_generator:
                # 将执行进度数据发生给前端
                yield self._format_sse_data(info)
                await asyncio.sleep(0.1)

    def _format_sse_data(self, data: Dict) -> str:
        """格式化sse数据"""
        data['timestamp'] = time.time()
        data['session_id'] = self.session_id
        json_data = json.dumps(data, ensure_ascii=False)
        return f'data: {json_data}\n\n'


@app.post('/agent/execute/stream', summary='调用Agent执行任务', tags=['AI智能体'])
async def execute_agent_stream(request: StreamRequest):
    """处理agent调用的请求方法"""
    user_input = request.user_input
    user_id = request.user_id
    # 创建一个会话id
    session_id = request.session_id or str(uuid.uuid4())
    # 创建一个agent执行器
    agent = ApiAgentExecutor(user_input=user_input, user_id=user_id,
session_id=session_id)
    # 执行agent，获得一个异步生成器
    async_generator = agent.execute_agent()
    # 返回结果
    return StreamingResponse(
        async_generator,
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "content-Type": "text/event-stream",
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Methods": "*",
```

```
            "Access-Control-Allow-Headers": "*",
        }
    )


if __name__ == '__main__':
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

- **agent.py**

```
# @Author  : 木森
# @weixin: python771
"""
自定义agnet(自己去控制agent的决策流程)的实现思路:

1、意图识别:对于用户输入的内容,我们需要精准的识别用户想干什么
    比如:
        帮我查询宠物电商项目的所有接口
            - 应该使用InterfaceQueryTool工具去实现
        帮我生成web测试平台,用户注册接口的用例
            - 应该使用InterfaceQueryTool工具去查询接口
            - 使用CompleteCaseGenerationInput去进行用例生成
        帮我生成web测试平台,用户注册接口的用例,并且执行生成的用例
            - 应该使用InterfaceQueryTool工具去查询接口
            - 使用CompleteCaseGenerationInput去进行用例生成
            - 使用CaseExecutionTool去执行测试用例
        执行web测试平台所有可用的用例。
            - 使用StructuredTestCaseQueryTool去查询出来web测试平台所有可用的用例
            - 使用CaseExecutionTool去执行测试用例

2、对于用户操作的需求实现,进行任务编排序列表。
    - 处理各个任务步骤之间的数据传递(agent的记忆去存储相关的工具执行的参数)
    - 哪个工具先执行,哪个工具后执行
    - 确认任务步骤执行的前置条件是否充分(所需参数是否完整)

3、分步调度执行任务
    - function calling的模式去调用(可以精确的智能构建工具执行所需的参数)

"""
import asyncio
import time

from agents.models import deepseek_r1_llm, dpr1_0528, qwen3_llm, deepseek_v3_llm
from prompts.agent_prompt.intent_recognition import intent_recognition_prompt
from tools.case_run.case_run import CaseExecutionTool
from tools.data_query.db_query_tools import InterfaceQueryTool,
BaseTestCaseQueryTool, StructuredTestCaseQueryTool
from tools.data_query.vec_retriever_tool import VectorRetrieverTool
from tools.generator_case.all_case_strutured_tool import BatchStructuredCaseTool
from tools.generator_case.base_case_generation_tool import BaseCaseGeneratorTool
from tools.generator_case.compete_case_generation_tool import
CompleteCaseGenerationTool
from tools.generator_case.one_case_strutured_tool import
OneStructuredCaseGenerationTool
from pydantic import BaseModel, Field
from typing import Dict, List
```

```python
from langchain_core.output_parsers import JsonOutputParser
from langgraph.store.memory import InMemoryStore
import uuid


class IntentRecognitionResult(BaseModel):
    """意图识别结果模型"""
    intent_type: str = Field(..., description="具体的意图类型")
    task_category: str = Field(..., description="任务大类")
    confidence: float = Field(..., ge=0.0, le=1.0, description="置信度")
    primary_tool: str = Field(..., description="主要使用的工具名称")
    tool_chain: List = Field(default_factory=list, description="工具调用链路，按顺序列出")
    is_batch_operation: bool = Field(False, description="是否为批量操作")
    scope: str = Field(..., description="操作范围")
    parameters: Dict = Field(..., description="参数信息")
    missing_params: List = Field(default_factory=list, description="缺失的关键参数")
    execution_plan: List = Field(default_factory=list, description="执行计划")
    workflow_type: str = Field(..., description="工作流类型")
    estimated_time: str = Field(..., description="预估执行时间")
    risk_level: str = Field(..., description="风险等级")


# 定义记忆处理的类
class MemoryHandler:
    def __init__(self, user_id: str, session_id: str):
        self.memory = InMemoryStore()
        self.user_id = user_id
        self.session_id = session_id
        self.namespace = (self.user_id, self.session_id)

    def save_to_memory(self, message: dict):
        """保存对话记录到记忆中"""
        try:
            uid = str(uuid.uuid4())
            self.memory.put(self.namespace, uid, message)
        except Exception as e:
            print("记忆存储失败：", e)

    def get_memory(self):
        """获取历史执行的记忆数据"""
        # 获取命名空间所有的记忆
        memory = self.memory.search(self.namespace)
        all_memory = []
        for item in memory:
            all_memory.append(item.value)
        return all_memory[-5:]


# 定义意图识别类
class IntentRecognition:
    intent_parser = JsonOutputParser(pydantic_schema=IntentRecognitionResult)

    def __init__(self, user_input, tools_info):
        """
        :param user_input: 用户输入的内容
        :param tools_info: 可用的工具信息
```

```
        """
        self.user_input = user_input
        self.tools_info = tools_info
        # 识别出来的意图结果
        self.intent_result = None

    def intent_recognition(self):
        """意图识别函数"""
        max_retries = 3
        for count in range(max_retries):
            try:
                tools_info = self.tools_info
                chain = intent_recognition_prompt | deepseek_v3_llm |
self.intent_parser
                # 进行意图识别
                intent_result = chain.invoke({"user_input": self.user_input,
"tool_info": tools_info})
                self.intent_result = intent_result
                print("意图识别结果:", intent_result)
                return intent_result
            except Exception as e:
                print(f"第{count + 1}意图识别失败: ", e)
                time.sleep(1)


# 工具执行器
class ToolExecutor:
    tools = {
        # 数据查询工具
        "vector_retriever": VectorRetrieverTool(),
        "interface_db_query": InterfaceQueryTool(),
        "base_test_case_query": BaseTestCaseQueryTool(),
        "structured_test_case_query": StructuredTestCaseQueryTool(),
        # 用例生成工具
        "complete_case_generation": CompleteCaseGenerationTool(),
        "base_case_generation": BaseCaseGeneratorTool(),
        "batch_structured_case": BatchStructuredCaseTool(),
        "one_structured_case_generation": OneStructuredCaseGenerationTool(),
        # 用例执行工具
        "case_execution": CaseExecutionTool(),
    }

    @classmethod
    def get_tools_info(cls):
        """获取工具信息"""
        tools_info = []
        for tool in cls.tools.values():
            tools_info.append({
                "name": tool.name,
                "description": tool.description,
                "parameters": tool.args_schema.model_json_schema() if
tool.args_schema else None
            })
        return tools_info

    def __init__(self, user_input, memory, intent_result, task_queue):
        """
        :param user_input:
```

```
        :param tools:
        :param memory:
        :param intent_result:
        """
        self.user_input = user_input
        self.memory = memory
        self.intent_result = intent_result
        self.task_queue = task_queue
        self.llm_with_tools =
deepseek_v3_llm.bind_tools(list(self.tools.values()))
        # 保存结构化的用例数据
        self.cases = []
        # 保存接口列表
        self.interfaces = []

    def func_calling_tool_execution(self, task):
        """
        func_calling工具执行
        完善了工具参数确实，function calling无法正确是被的问题
        :param task:
        :return:
        """
        result = None
        max_retries = 3
        for count in range(max_retries):
            func_prompt = self.get_func_calling_prompt(task)
            # 使用大模型function calling模型去根据上下文构建工具调用
            # tools_result = self.llm_with_tools.invoke(func_prompt)
            tools_result =
deepseek_v3_llm.bind_tools([self.tools.get(task.get('tool_name'))]).invoke(func_
prompt)
            # 获取要执行的工具
            if hasattr(tools_result, "tool_calls") and tools_result.tool_calls:
                # 获取要执行的工具
                for tool in tools_result.tool_calls:
                    # 获取工具名称
                    tool_name = tool.get('name', '')
                    # 获取工具参数
                    tool_args = tool.get('args', '')
                    # 实际的工具执行
                    result = self.tools[tool_name].invoke(tool_args)
                    # 判断工具执行的返回的是否为结构化用例的数据
                    if task.get('tool_name') in ["complete_case_generation",
                                                 "structured_test_case_query",
                                                 "batch_structured_case",

"one_structured_case_generation"]:
                        # 保存结构化的用例数据
                        self.cases = result
                    elif task.get('tool_name') == "interface_db_query":
                        self.interfaces = result
                    else:
                        self.memory.save_to_memory({
                            "role": "assistant",
                            "content": result,
                            "type": "tool_execution",
                            "tool_name": task.get('tool_name'),
                        })
```

```
                print(f"步骤{task.get('action')}执行完毕")
                print(f"工具{tool_name}执行的结果：", result)
        else:
            print("工具识别失败，请检查工具识别结果：", tools_result.content)
            data = input("请按照错误提示补充相关信息：")
            # 将补充的内容加入到记忆中
            self.memory.save_to_memory({
                "role": "assistant",
                "content": data,
                "type": "tool_execution",
                "tool_name": task.get('tool_name'),
            })
        return result

    def get_func_calling_prompt(self, task):
        """
        构建func_calling的提示词的组成：
            执行的任务
            上下文(从记忆中获取)
                初始需求：
                意图识别的结果：
                历史工具执行的结果
        :param tools_result:
        :return:
        """
        prompt = f"""
          用户需求：{self.user_input}
          当前任务：{task.get('action')}
          本次调用的工具：{task.get('tool_name')}
          可用的参数数据列表：
          {self.intent_result.get('parameters')}
          历史执行的数据
          {self.memory.get_memory()}
          请调用工具{task.get('tool_name')}来完成任务
          """
        return prompt

    async def task_execution(self):
        """任务执行的函数"""
        # 获取任务的步骤总数
        total_steps = len(self.task_queue)
        i = 0
        while self.task_queue:
            i += 1
            # 本轮工具执行的结果
            result = None
            # 取出要执行的任务
            task = self.task_queue.pop()
            # 开始执行步骤
            yield {
                "type": "step_start",
                "step_info": {
                    "step": task.get('step'),
                    "action": task.get("action"),
                    "tool_name": task.get("tool_name")
                },
                "step_number": task.get('step'),
                "total_steps": total_steps,
```

```
                "progress": (i / total_steps) * 100 + 10  # 计算执行的进度
            }
            await asyncio.sleep(0.1)
            # 设置单个任务步骤的执行失败重试机制
            max_retries = 3
            for count in range(max_retries):
                try:
                    if task.get('tool_name') == "case_execution":
                        yield self.return_tools_start_info(task, i)
                        # 手动调用执行测试用例的工具
                        result = CaseExecutionTool().invoke({"test_data":
self.cases})
                        yield self.return_tools_end_info(task, result)
                        # 将工具执行的结果加入到记忆中
                        self.memory.save_to_memory({
                            "role": "assistant",
                            "content": result,
                            "type": "tool_execution",
                            "tool_name": task.get('tool_name'),
                        })
                    elif task.get('is_loop') and task.get('tool_name') ==
'complete_case_generation':
                        # 是否为需要循环批量去执行的工具
                        for interface in self.interfaces:
                            print(f"==================正在为用例生成测试用例
==================")
                            # 把当前要生成的接口添加到记忆中
                            self.memory.save_to_memory({
                                "role": "system",
                                "content": f"接口信息为：{interface}"
                            })
                            yield self.return_tools_start_info(task, i)
                            result = self.func_calling_tool_execution(task)
                            yield self.return_tools_end_info(task, result)
                    else:
                        if self.interfaces:
                            # 添加接口信息到记忆中
                            self.memory.save_to_memory({
                                "role": "system",
                                "content": f"接口信息为：{self.interfaces[-1]}"
                            })
                        yield self.return_tools_start_info(task, i)
                        self.func_calling_tool_execution(task)
                        yield self.return_tools_end_info(task, result)
                    # 结束循环
                    break
                except Exception as e:
                    print(f"工具【{task.get('tool_name')}】第{count + 1}次执行失
败：", e)
                    time.sleep(1)
            # 步骤执行完毕
            yield {
                "type": "step_complete",
                "step_info": {
                    "step": task.get('step'),
                    "action": task.get("action"),
                    "tool_name": task.get("tool_name")
                },
```

```
                "step_number": task.get('step'),
                "result": f"步骤 {i} 执行完成",
                "progress": (i / total_steps) * 100 + 10
            }
            await asyncio.sleep(0.1)

    @staticmethod
    def return_tools_start_info(task, i):
        return {
            "type": "tool_interaction",
            "interaction_type": "start",
            "message": f"开始执行工具: {task.get('tool_name')}",
            "data": {
                "tool_name": task.get('tool_name'),
                "action": task.get('action'),
                "step_number": i
            }
        }

    @staticmethod
    def return_tools_end_info(task, result):
        return {
            "type": "tool_interaction",
            "interaction_type": "result",
            "message": f"工具执行完成: {task.get('tool_name')}",
            "data": {
                "tool_name": task.get('tool_name'),
                "result": result,
                "success": True if result else False,
            }
        }
```

3、前端源码

```
<!DOCTYPE html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>接口测试智能体</title>
    <style>
      * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
      }

      body {
        font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

        min-height: 100vh;
        padding: 20px;
```

```
}

.container {
  max-width: 1800px;
  margin: 0 auto;
  background: white;
  border-radius: 15px;
  box-shadow: 0 20px 40px rgba(0,0,0,0.1);
  overflow: hidden;
}

.header {
  background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
  color: white;
  padding: 30px;
  text-align: center;
}

.header h1 {
  font-size: 2.5em;
  margin-bottom: 10px;
  font-weight: 300;
}

.header p {
  font-size: 1.1em;
  opacity: 0.9;
}

.main-content {
  padding: 30px;
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 30px;
}

.input-section {
  background: #f8f9ff;
  padding: 25px;
  border-radius: 10px;
  border: 1px solid #e1e8ff;
}

.output-section {
  background: #f8f9ff;
  padding: 25px;
  border-radius: 10px;
  border: 1px solid #e1e8ff;
}

.section-title {
  font-size: 1.3em;
  color: #333;
  margin-bottom: 20px;
  font-weight: 600;
}

.input-group {
```

```
  margin-bottom: 20px;
}

label {
  display: block;
  margin-bottom: 8px;
  color: #555;
  font-weight: 500;
}

input[type="text"], textarea {
  width: 100%;
  padding: 12px;
  border: 2px solid #e1e8ff;
  border-radius: 8px;
  font-size: 14px;
  transition: border-color 0.3s ease;
}

input[type="text"]:focus, textarea:focus {
  outline: none;
  border-color: #4facfe;
}

textarea {
  resize: vertical;
  min-height: 120px;
}

.button-group {
  display: flex;
  gap: 10px;
  margin-top: 20px;
}

button {
  padding: 12px 24px;
  border: none;
  border-radius: 8px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
}

.btn-primary {
  background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
  color: white;
}

.btn-primary:hover {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(79, 172, 254, 0.4);
  }

  .btn-secondary {
      background: #6c757d;
      color: white;
```

```
        }

        .btn-secondary:hover {
            background: #5a6268;
        }

        .btn-danger {
            background: #dc3545;
            color: white;
        }

        .btn-danger:hover {
            background: #c82333;
        }

        .status-bar {
            background: #e9ecef;
            padding: 15px;
            border-radius: 8px;
            margin-bottom: 20px;
            border-left: 4px solid #4facfe;
        }

        .status-text {
            font-weight: 500;
            color: #333;
        }

        .progress-bar {
            width: 100%;
            height: 8px;
            background: #e9ecef;
            border-radius: 4px;
            margin-top: 10px;
            overflow: hidden;
        }

        .progress-fill {
            height: 100%;
            background: linear-gradient(90deg, #4facfe, #00f2fe);
            width: 0%;
            transition: width 0.5s ease;
            border-radius: 4px;
        }

        .log-container {
            background: #1e1e1e;
            color: #f8f8f2;
            padding: 20px;
            border-radius: 8px;
            height: 400px;
            overflow-y: auto;
            font-family: 'Consolas', 'Monaco', monospace;
            font-size: 13px;
            line-height: 1.6;
        }

        .log-entry {
```

```
            margin-bottom: 8px;
            padding: 8px;
            border-radius: 4px;
            border-left: 3px solid transparent;
        }

        .log-start {
            background: rgba(40, 167, 69, 0.1);
            border-left-color: #28a745;
        }

        .log-progress {
            background: rgba(255, 193, 7, 0.1);
            border-left-color: #ffc107;
        }

        .log-intent {
            background: rgba(0, 123, 255, 0.1);
            border-left-color: #007bff;
        }

        .log-step {
            background: rgba(108, 117, 125, 0.1);
            border-left-color: #6c757d;
        }

        .log-complete {
            background: rgba(40, 167, 69, 0.1);
            border-left-color: #28a745;
        }

        .log-error {
            background: rgba(220, 53, 69, 0.1);
            border-left-color: #dc3545;
        }

        .log-llm {
            background: rgba(138, 43, 226, 0.1);
            border-left-color: #8a2be2;
        }

        .log-tool {
            background: rgba(255, 140, 0, 0.1);
            border-left-color: #ff8c00;
        }

        .timestamp {
            color: #6c757d;
            font-size: 11px;
        }

        .json-data {
            background: rgba(108, 117, 125, 0.1);
            padding: 10px;
            border-radius: 4px;
            margin-top: 5px;
            font-size: 11px;
            overflow-x: auto;
```

```
}

.intent-result {
    background: #e7f3ff;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 15px;
    border: 1px solid #b3d9ff;
}

.intent-title {
    font-weight: 600;
    color: #0056b3;
    margin-bottom: 10px;
}

.confidence-bar {
    background: #e9ecef;
    height: 6px;
    border-radius: 3px;
    overflow: hidden;
    margin-top: 5px;
}

.confidence-fill {
    height: 100%;
    background: linear-gradient(90deg, #dc3545, #ffc107, #28a745);
    transition: width 0.5s ease;
}

.execution-plan {
    margin-top: 15px;
}

.plan-step {
    background: white;
    padding: 10px;
    margin: 5px 0;
    border-radius: 5px;
    border-left: 3px solid #4facfe;
    font-size: 13px;
}

.step-number {
    font-weight: 600;
    color: #4facfe;
}

@media (max-width: 768px) {
    .main-content {
        grid-template-columns: 1fr;
        gap: 20px;
    }

    .header h1 {
        font-size: 2em;
    }
}
```

```
        .loading-spinner {
            display: inline-block;
            width: 20px;
            height: 20px;
            border: 3px solid #f3f3f3;
            border-top: 3px solid #4facfe;
            border-radius: 50%;
            animation: spin 1s linear infinite;
        }

        @keyframes spin {
            0% { transform: rotate(0deg); }
            100% { transform: rotate(360deg); }
        }

        .hidden {
            display: none;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>🤖 接口测试AI智能体</h1>
            <p>基于接口文档，使用AI驱动进行接口测试的智能体</p>
        </div>

        <div class="main-content">
            <!-- 输入区域 -->
            <div class="input-section">
                <h2 class="section-title">📝 输入任务</h2>

                <div class="input-group">
                    <label for="userInput">用户输入:</label>
                    <textarea id="userInput" placeholder="请输入您的需求，例如：查询web测试平台用户注册的接口，生成测试用例">查询web测试平台用户注册的接口，生成测试用例
</textarea>
                </div>

                <div class="input-group">
                    <label for="userId">用户ID:</label>
                    <input type="text" id="userId" placeholder="可选，用于会话管理" value="demo_user">
                </div>

                <div class="button-group">
                    <button id="executeBtn" class="btn-primary">
                        🚀 开始执行
                    </button>
                    <button id="stopBtn" class="btn-danger hidden">
                        ⏹ 停止执行
                    </button>
                    <button id="clearBtn" class="btn-secondary">
                        🗑 清空日志
                    </button>
                </div>
            </div>
```

```
            <!-- 输出区域 -->
            <div class="output-section">
                <h2 class="section-title">📊 执行状态</h2>

                <div class="status-bar">
                    <div class="status-text" id="statusText">等待执行...</div>
                    <div class="progress-bar">
                        <div class="progress-fill" id="progressFill"></div>
                    </div>
                </div>

                <!-- 意图识别结果 -->
                <div id="intentResult" class="intent-result hidden">
                    <div class="intent-title">🎯 意图识别结果</div>
                    <div id="intentContent"></div>
                </div>

                <!-- 执行计划 -->
                <div id="executionPlan" class="execution-plan hidden">
                    <div class="intent-title">📋 执行计划</div>
                    <div id="planContent"></div>
                </div>
            </div>
        </div>

        <!-- 日志区域 -->
        <div style="padding: 0 30px 30px;">
            <h2 class="section-title">📜 执行日志</h2>
            <div class="log-container" id="logContainer">
                <div class="log-entry">
                    <span class="timestamp">[等待中]</span> 准备就绪，点击"开始执
行"按钮开始...
                </div>
            </div>
        </div>
    </div>

    <script>
        class AgentStreamClient {
            constructor() {
                this.eventSource = null;
                this.isExecuting = false;
                this.sessionId = null;

                this.initializeElements();
                this.bindEvents();
            }

            initializeElements() {
                this.elements = {
                    executeBtn: document.getElementById('executeBtn'),
                    stopBtn: document.getElementById('stopBtn'),
                    clearBtn: document.getElementById('clearBtn'),
                    userInput: document.getElementById('userInput'),
                    userId: document.getElementById('userId'),
                    statusText: document.getElementById('statusText'),
                    progressFill: document.getElementById('progressFill'),
```

```
                logContainer: document.getElementById('logContainer'),
                intentResult: document.getElementById('intentResult'),
                intentContent: document.getElementById('intentContent'),
                executionPlan: document.getElementById('executionPlan'),
                planContent: document.getElementById('planContent')
            };
        }

        bindEvents() {
            this.elements.executeBtn.addEventListener('click', () =>
this.startExecution());
            this.elements.stopBtn.addEventListener('click', () =>
this.stopExecution());
            this.elements.clearBtn.addEventListener('click', () =>
this.clearLog());
        }

        async startExecution() {
            const userInput = this.elements.userInput.value.trim();
            if (!userInput) {
                alert('请输入您的需求');
                return;
            }

            this.isExecuting = true;
            this.sessionId = this.generateSessionId();

            // 更新UI状态
            this.elements.executeBtn.classList.add('hidden');
            this.elements.stopBtn.classList.remove('hidden');
            this.elements.statusText.textContent = '正在连接...';
            this.updateProgress(0);

            // 隐藏之前的结果
            this.elements.intentResult.classList.add('hidden');
            this.elements.executionPlan.classList.add('hidden');

            try {
                // 创建EventSource连接
                const url = 'http://localhost:8000/agent/execute/stream';

                // 使用POST请求启动流式连接
                const response = await fetch(url, {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json',
                    },
                    body: JSON.stringify({
                        user_input: userInput,
                        user_id: this.elements.userId.value || 'anonymous',
                        session_id: this.sessionId
                    })
                });

                if (!response.ok) {
                    throw new Error(`HTTP ${response.status}:
${response.statusText}`);
                }
```

```
                // 读取流式响应
                const reader = response.body.getReader();
                const decoder = new TextDecoder();

                while (this.isExecuting) {
                    const { done, value } = await reader.read();

                    if (done) break;

                    const chunk = decoder.decode(value);
                    const lines = chunk.split('\n');

                    for (const line of lines) {
                        if (line.startsWith('data: ')) {
                            try {
                                const data = JSON.parse(line.substring(6));
                                this.handleStreamData(data);
                            } catch (e) {
                                console.error('解析数据失败:', e, line);
                            }
                        }
                    }
                }

            } catch (error) {
                console.error('执行失败:', error);
                this.addLog('error', `连接失败: ${error.message}`);
            } finally {
                this.stopExecution();
            }
        }

        handleStreamData(data) {
            console.log('收到数据:', data);
            switch (data.type) {
                case 'start':
                    this.addLog('start', data.message);
                    this.elements.statusText.textContent = '开始执行...';
                    break;

                case 'progress':
                    this.addLog('progress', `${data.message}
(${data.progress}%)`);
                    this.elements.statusText.textContent = data.message;
                    this.updateProgress(data.progress);
                    break;

                case 'intent_result':
                    this.addLog('intent', '意图识别完成');
                    this.displayIntentResult(data.data);
                    this.updateProgress(data.progress);
                    break;

                case 'llm_interaction':
                    if (data.interaction_type === 'request') {
                        this.addLog('llm', `🤖 ${data.message}`, data.data);
                    } else if (data.interaction_type === 'response') {
```

```
                        this.addLog('llm', `☑ ${data.message}`, data.data);
                    }
                    break;

                case 'tool_interaction':
                    if (data.interaction_type === 'start') {
                        this.addLog('tool', `🔧 ${data.message}`,
data.data);
                    } else if (data.interaction_type === 'call') {
                        this.addLog('tool', `⚡ ${data.message}`,
data.data);
                    } else if (data.interaction_type === 'result') {
                        this.addLog('tool', `☑ ${data.message}`,
data.data);
                    } else if (data.interaction_type === 'error') {
                        this.addLog('error', `❌ ${data.message}`,
data.data);
                    }
                    break;

                case 'execution_plan':
                    this.addLog('step', `执行计划已生成 (${data.data.length} 个
步骤)`);
                    this.displayExecutionPlan(data.data);
                    this.updateProgress(data.progress);
                    break;

                case 'step_start':
                    this.addLog('step', `开始执行步骤
${data.step_number}/${data.total_steps}: ${data.step_info.action}`);
                    this.elements.statusText.textContent = `执行步骤
${data.step_number}/${data.total_steps}`;
                    this.updateProgress(data.progress);
                    break;

                case 'step_complete':
                    this.addLog('complete', `步骤 ${data.step_number} 完成:
${data.result}`);
                    this.updateProgress(data.progress);
                    break;

                case 'complete':
                    this.addLog('complete', data.message);
                    this.elements.statusText.textContent = '执行完成!';
                    this.updateProgress(100);
                    if (data.data) {
                        this.addLog('complete', '执行结果:', data.data);
                    }
                    this.stopExecution();
                    break;

                case 'error':
                    this.addLog('error', data.message);
                    this.elements.statusText.textContent = '执行失败';
                    if (data.error) {
                        this.addLog('error', '错误详情:', data.error);
                    }
                    this.stopExecution();
```

```
                    break;
                }
            }

            displayIntentResult(intentData) {
                this.elements.intentResult.classList.remove('hidden');

                const confidence = intentData.confidence || 0;
                const confidencePercent = Math.round(confidence * 100);

                this.elements.intentContent.innerHTML = `
                    <div><strong>意图类型:</strong> ${intentData.intent_type ||
'未知'}</div>
                    <div><strong>任务类别:</strong> ${intentData.task_category ||
'未知'}</div>
                    <div><strong>主要工具:</strong> ${intentData.primary_tool ||
'未知'}</div>
                    <div><strong>置信度:</strong> ${confidencePercent}%</div>
                    <div class="confidence-bar">
                        <div class="confidence-fill" style="width:
${confidencePercent}%"></div>
                    </div>
                `;
            }

            displayExecutionPlan(planData) {
                this.elements.executionPlan.classList.remove('hidden');

                const planHtml = planData.map((step, index) => `
                    <div class="plan-step">
                        <span class="step-number">步骤 ${index + 1}:</span>
                        ${step.action}
                        <div style="font-size: 11px; color: #666; margin-top:
3px;">
                            工具: ${step.tool_name}
                        </div>
                    </div>
                `).join('');

                this.elements.planContent.innerHTML = planHtml;
            }

            stopExecution() {
                this.isExecuting = false;

                if (this.eventSource) {
                    this.eventSource.close();
                    this.eventSource = null;
                }

                // 更新UI状态
                this.elements.executeBtn.classList.remove('hidden');
                this.elements.stopBtn.classList.add('hidden');
            }

            updateProgress(percent) {
                this.elements.progressFill.style.width = `${percent}%`;
            }
```

```
            addLog(type, message, data = null) {
                const timestamp = new Date().toLocaleTimeString();
                const logEntry = document.createElement('div');
                logEntry.className = `log-entry log-${type}`;

                let content = `<span class="timestamp">[${timestamp}]</span>
${message}`;

                if (data) {
                    content += `<div class="json-data">${JSON.stringify(data,
null, 2)}</div>`;
                }

                logEntry.innerHTML = content;
                this.elements.logContainer.appendChild(logEntry);
                this.elements.logContainer.scrollTop =
this.elements.logContainer.scrollHeight;
            }

            clearLog() {
                this.elements.logContainer.innerHTML = `
                    <div class="log-entry">
                        <span class="timestamp">[清空]</span> 日志已清空，准备新的执
行...
                    </div>
                `;

                // 隐藏结果区域
                this.elements.intentResult.classList.add('hidden');
                this.elements.executionPlan.classList.add('hidden');

                // 重置状态
                this.elements.statusText.textContent = '等待执行...';
                this.updateProgress(0);
            }

            generateSessionId() {
                return 'session_' + Date.now() + '_' +
Math.random().toString(36).substr(2, 9);
            }
        }

        // 初始化应用
        document.addEventListener('DOMContentLoaded', () => {
            new AgentStreamClient();
        });
    </script>
</body>
</html>
```